systems

MDPI

Article

Specifying Simulation Commitments Early in System Design: Introduction to the EFSUT Methodology

Bernard P. Zeigler

RTSync Corp., 6909 W. Ray Road, Chandler, AZ 85226, USA; zeigler@rtsync.com

Abstract

Engineering programs increasingly rely on simulation for early design decisions, yet a simulation's obligations—the specific questions it must answer and observations it must provide—are often left implicit until late in development. The Experimental Frame–System Under Test (EFSUT) methodology addresses this gap by requiring these commitments to be specified explicitly early in system design. EFSUT organizes simulation around four core elements—questions, experimental conditions, models, and results—and defines formal relations that clarify how tests are derived and which models can meaningfully be evaluated. We demonstrate the methodology in two contrasting domains, showing how an informally stated motivating question leads to a structured sequence of experimental conditions and guides model selection and interpretation. Compared with existing approaches to model adequacy and digital-engineering traceability, EFSUT provides a clear, question-driven foundation that links stakeholder intent to model evaluation in a transparent, defensible manner. This approach is particularly valuable for aligning simulation practice in multi-model and system-of-systems contexts. Future work includes the automated derivation of Experimental Frames, integration with digital-thread toolchains, and more broadly, development of a lifecycle-spanning, formalized question-driven framework to support evaluation, optimization, and adaptation where traditional requirement-based methods fall short.

Keywords: digital engineering; DEVS; SES; experimental frames; morphisms; modular design; IoT; simulation frameworks

Check for updates

Academic Editors: Lawrence Dale Thomas, Ana Wooley and Hanumanthrao Kannan

Received: 17 April 2026
Revised: 3 June 2026
Accepted: 12 June 2026
Published: 16 June 2026

Copyright: © 2026 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Modeling and simulation have long served as foundational tools for understanding, predicting, and controlling the behavior of complex systems. Within this tradition, the DEVS (Discrete Event System Specification) formalism provides a rigorous framework for representing system dynamics in a modular and hierarchical manner, while Experimental Frames (EFs) specify the observational and contextual conditions under which those models are exercised. Together, they form a powerful basis for simulation-based inquiry.

As systems grow in scale and heterogeneity, and as simulation becomes central to early-stage engineering decisions, verification activities, and certification arguments, the commitments that simulations must satisfy—what questions they must answer, what observables they must expose, and what conditions they must operate under—must be made explicit far earlier in the design process than is typical today.

In this paper, we define a new methodology, called EFSUT (Experimental Frame-System Under Test), that addresses this gap by introducing a structured chain that begins

with a question, proceeds through a derived Experimental Frame, and culminates in the evaluation of a model under that frame.

The approach depends on the System Entity Structure (SES)/DEVS [1] formalism that enables the systematic generation of model variants and supports compositional reasoning about model structure while leaving open how stakeholder questions should be mapped to specific configurations or evaluation tests.

Traditional DEVS practice presumes that the modeler already knows what to test and how to interpret the results. In practice, however, stakeholder questions must be translated into testable observational structures, and models must be evaluated only within the scope of those structures.

Without a principled mechanism for specifying these simulation commitments early, simulation studies risk becoming ad hoc, difficult to replicate, or misaligned with stakeholder concerns.

The EFSUT (Experimental Frame–System Under Test) methodology addresses this gap by introducing a structured chain that begins with a Question, proceeds through a derived Experimental Frame, and culminates in the evaluation of a model under that frame. This chain is governed by three relations—Derivability, Applicability, and Morphisms—which together organize the space of frames and models into a coherent, question-driven structure. Derivability orders frames by how they refine or coarsen the concerns expressed in a Question. Applicability determines whether a given frame can be meaningfully interpreted on a given model. Morphisms relate models across abstraction levels, enabling justified simplification or refinement. These relations provide the conceptual infrastructure needed to ensure that simulation results are traceable, auditable, and defensible.

To introduce the methodology, we begin with a simple timing question that gives rise to a hierarchy of Experimental Frames, each representing a different level of observational granularity. Deterministic, stochastic, and unbounded models behave differently under these frames, and the EFSUT relations clarify these differences. This example demonstrates how a well-structured relational framework can bring order and interpretability to the inherently multifaceted process of simulation model development.

The remainder of the paper demonstrates the methodology across two heterogeneous DEVS-based domains: a naval susceptibility engagement model and a dynamic-structure UAV swarm model. Each domain yields a question-driven EF chain grounded in its motivating operational concern. A cross-domain question then motivates a combined Experimental Frame requiring system-of-systems observables, which is satisfied only by a notional base model that integrates both domains. This naval–swarm example illustrates how EFSUT scales from single-model reasoning to multi-model, multi-frame, system-of-systems analysis, providing a coherent and formally grounded approach to simulation-based decision support.

After reviewing the background needed, the final sections situate EFSUT within the broader landscape of model adequacy assessment, digital-engineering traceability, and contract-based system specification. We then outline future work on automated EF derivation, digital-thread integration, and formal support for dynamic-structure and adaptive models.

2. Basic Relations

The EFSUT methodology works on the core elements: *Questions*, *Experimental Frames* (EFs), and *Models* derived from questions that articulate the stakeholder's concerns. The relations of *derivability* among frames, *morphism* among models, and *applicability* of frames to models together form a disciplined workflow that links requirements to simulation results. These relations, summarized in Table 1, provide the conceptual infrastructure needed

to ensure that simulation results are traceable, auditable, and defensible. They allow Questions to be systematically transformed into executable tests, ensure that models are evaluated only under appropriate conditions, and support principled comparison among models. In this way, EFSUT extends the DEVS framework from a modeling formalism into a full methodology for simulation-based reasoning.

Table 1. The three EFSUT relations: domains, definitions, and roles in organizing development and testing.

Relation	Domain	Formal Definition	Role in EFSUT Process
Derivability	Experimental Frames (E, E')	$E \leq E'$ iff E can be obtained from E' by the *selection*, %MDPI: Please confirm if the italics are necessary; if not, please remove them. The following highlights are the same. *restriction*, or *coarsening* of variables, admissible behaviors, or observables.	Organizes frames into a *partial order* (frame partial order). Shows how more restrictive tests derive from more inclusive ones. Supports systematic refinement of Questions into executable tests.
Applicability	Frame–Model pairs (E, M)	E is applicable to M iff the EF can be meaningfully interpreted on the model: matching input/output ports and interpretable EF semantics.	Determines which models can be tested under which frames. Separates *can run the test* from *passes the test*. Ensures that the EF–SUT coupling is well formed before simulation.
Morphism	Model–Model pairs (M, M')	$M \rightarrow M'$ iff M' preserves the behavioral structure of M at a given level of abstraction (exact or approximate).	Organizes models into abstraction hierarchies. Supports model simplification, refinement, and comparison. Ensures that results obtained from simpler models are justified by structural correspondence.

3. Related Work

Modeling and simulation research has long emphasized the need for explicit structures that connect system behavior, experimental conditions, and stakeholder concerns—see [2] for a detailed review. The DEVS formalism [1] provides a mathematically rigorous foundation for representing discrete-event systems through modular, hierarchical models. DEVS separates the specification of system dynamics from the specification of experimental conditions, which is a distinction later formalized through the concept of the Experimental Frame (EF) [1]. EFs define admissible inputs, observables, and termination conditions, and they have been used extensively in simulation-based analysis, verification, and design.

The System Entity Structure (SES) [1] complements DEVS by providing a declarative representation of model families, capturing decomposition, specialization, and configuration choices. SES–DEVS integration enables the systematic generation of model variants and supports compositional reasoning about model structure. However, SES does not prescribe how models should be evaluated nor how stakeholder questions should be mapped to specific configurations or tests.

Background

The Theory of Modeling and Simulation (TMS) emphasizes the need to explicitly relate a simulation model to the conditions under which it is exercised. Across decades of DEVS research, this principle has been developed through methodologies that progressively clarify the relationship between *models*, *their contexts*, and the *objectives that motivate their use*. The EFSUT methodology builds directly on this lineage.

Early work established the foundational separation between a system model and the environment in which it is tested [1]. Experimental Frames (EFs) were introduced as formal system specifications that define admissible inputs, observables, and termination conditions. Later work showed that simulation validity depends on the compatibility between the model and its admissible EFs. As summarized in [3], validation can be

assessed by "measuring the distance (compatibility) between the behavior specified by the user and the behavior of the simulation model," using I/O automata and multi-phase analysis to quantify mismatches.

As simulation objectives became more complex—particularly in hybrid and cyber-physical systems—researchers recognized the need for abstraction hierarchies within both models and frames. Reference [4] demonstrates that achieving a given analytical goal requires "necessary and consistent modeling abstractions in Experimental Frame components," which are organized as partially ordered sets of trace-based behavioral constraints. This perspective aligns with [5], which formalizes the "dual relationship between simulation models and their context" and introduces a specification hierarchy for EFs that mirrors the DEVS system hierarchy.

Parallel developments in multiformalism and multilevel modeling further reinforced the need for explicit relationships among models and frames. The ecosystem example in [6] shows how multiple models—differential equation, discrete-event, and hybrid—can be organized into "planes of abstraction" with Experimental Frames serving as the connective tissue that allows simpler models to verify more complex ones and vice versa. This work highlights the importance of model morphisms, frame derivability, and cross-model consistency—all central concepts in EFSUT. Because of its extensive elucidation of concepts and in depth application, this book chapter is available for the reader's convenience in a Supplemental File associated with this article (open access with publisher's permission).

The organization of partial models introduced another key insight: large-scale systems require a plurality of models, each answering only a specific subset of questions. Ref. [7] explicitly describes a theoretical foundation in which "Experimental Frames and models are organized by derivability and morphism relations" and proposes modules for model simplification and elaboration. This relational view anticipates the EFSUT emphasis on structuring the space of questions, frames, and models.

More recent work has focused on operationalizing these ideas in software toolchains. Reference [8] extended earlier work to support the automated generation of EF components—acceptors, generators, transducers—and integrated error models for software quality evaluation. Similarly, ref. [9] develops a methodology for model discovery and composition using SES ontologies, characterizing models independently of their simulators and frames to support repository-based retrieval and reuse. Ref. [3] complements this by proposing an Internet-based repository for DEVS models and their Experimental Frames, enabling systematic curation and sharing.

The EFSUT methodology synthesizes these strands into a unified, question-driven workflow and provides a disciplined, traceable workflow to link stakeholder requirements to executable DEVS models. It begins with questions, deriving Experimental Frames through a partial order of observational commitments, and determining applicability constraints that govern which models can be meaningfully tested. This approach operationalizes decades of theoretical work on frames, compatibility, abstraction, and partial models while providing a practical pipeline for implementation in environments such as [10].

As will be discussed in Section 8, model-based testing (MBT) frameworks address the generation of test cases from formal models but typically assume a fixed test specification and do not provide a principled mechanism for deriving tests from stakeholder concerns. MBT approaches also lack a formal notion of applicability: they assume that the test suite is appropriate for the System Under Test rather than providing a relational structure that determines when a test is meaningful.

The EFSUT methodology builds on these foundations by introducing a relational layer that explicitly connects questions, frames, and models. Derivability organizes frames into a partial order, enabling the systematic refinement of questions into executable tests. Applicability determines whether a frame can be meaningfully interpreted on a model, ensuring

that EF–SUT couplings are well formed. Morphism relates models across abstraction levels, supporting justified simplification and refinement. Together, these relations extend DEVS and SES from modeling formalisms into a full methodology for simulation-based reasoning, bridging the gap between stakeholder intent and model evaluation.

The DEVS formalism, EF semantics, and the theoretical basis for context specification and abstraction hierarchies draw on established work in the literature [4,5,9].

4. Introduction to the Response-Time Example

To make the EFSUT methodology concrete, we develop a simple but representative *response-time example*. The purpose of the example is not to showcase a sophisticated system but rather to illustrate—using the smallest possible model—the full chain from stakeholder question to derived Experimental Frames, to accommodating models, and finally to interpretable results. This example serves as a running thread through the paper, while its complete DEVS specification is provided in Appendix A. The response-time example extends earlier EF-driven test structures and the minimal-model reasoning introduced in foundational EF and partial-model studies [7]. The example illustrates how the basic relations operate in practice. A simple timing question gives rise to a hierarchy of Experimental Frames, each representing a different level of observational granularity. Multiple models—deterministic, stochastic, and unbounded—can be evaluated under these frames, but their behaviors differ in ways that the applicability and morphism relations help clarify. Through this example, the EFSUT methodology demonstrates how a well-structured relational framework can bring order and interpretability to the inherently multifaceted process of simulation-based testing.

The motivating Question is straightforward:

> *Does the system produce output Y within a specified time bound* $T_{\max}$ *after receiving input X?*

This example is intentionally minimal: it involves a single input stimulus, a bounded response requirement, and a small family of Experimental Frames that differ only in their observational commitments.

Although simple, this question captures a pattern that appears across many domains—communication latency, sensor response, actuator delay, and more generally any requirement that constrains the time between stimulus and response. Because the question is so common, it provides an ideal setting for demonstrating how EFSUT structures the testing process.

Figure 1 illustrates the I/O Temporal Behavior and processing by the experimental frames for the response time example.

From the just stated question, we derive a family of Experimental Frames (Figure 2) that differ in how much they observe and how strictly they constrain the test. At the most inclusive level, a *full I/O frame* records all inputs and outputs without imposing any time bound. A more specific *response-time frame* emits a single input at time zero and waits up to $T_{\max}$ for the corresponding output. A third, *pass/fail frame*, coarsens the observation by discarding the measured delay and reporting only whether the system meets the time requirement of the question. As will be discussed soon, the concept of derivability induces a partial order over Experimental Frames. In this example, the three frames form a full order, and each can be derived from the next going from bottom to top in the figure (a full order is special case of a partial order). This illustrates how EFSUT organizes frames according to the observational commitments they impose.

On the model side, we consider three simple DEVS atomic models that represent different timing behaviors: a deterministic delay model, a stochastic delay model, and an unbounded model that may accept input but never respond. These models share the same

interface but differ in their internal timing assumptions. This allows us to demonstrate two key EFSUT relations: applicability, which determines whether a frame can be meaningfully interpreted on a model, and morphism, which relates pairs of models, for example the stochastic and deterministic delay models, while the unbounded model has no useful morphic relation to the others.

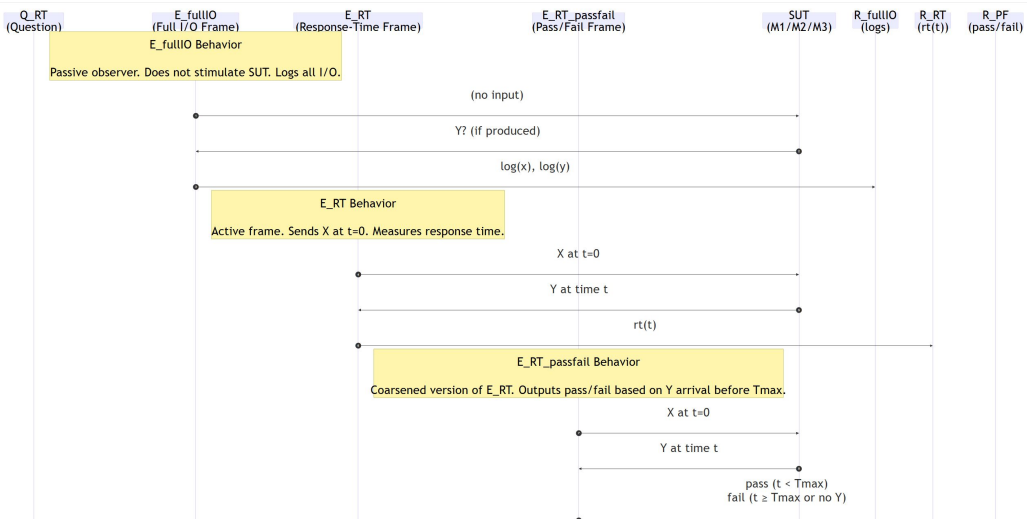


Figure 1. Question-related I/O temporal behavior and EF handling. The figure shows how a model behaves under the conditions specified by three different frames: (**top**) a full I/O frame where the informal notion of 'no input' is treated formally as a null segment—that is, a time-segment mapped to the single distinguished value null; (**middle**) a response-time frame that emits a single input at time zero and waits up to Tmax for the corresponding output; and (**bottom**) a pass/fail frame that further coarsens observation to report only whether the system met the time requirement.

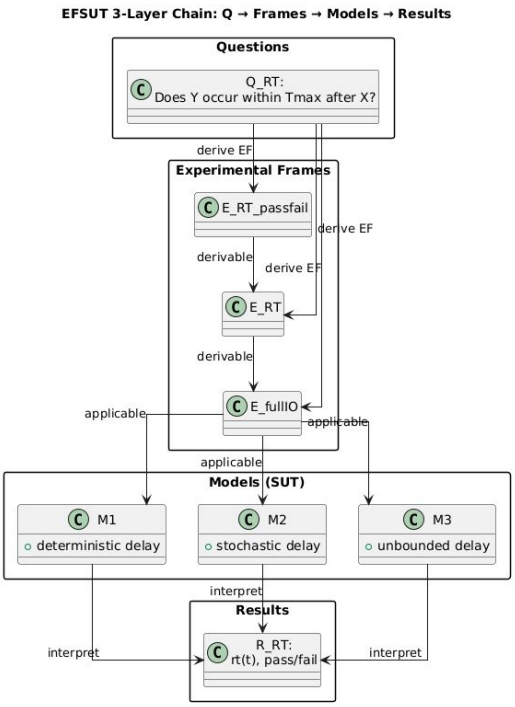


Figure 2. Minimal EFSUT relations.

By combining these frames and models, the response-time example illustrates the full EFSUT chain:

$$Q \longrightarrow E \longrightarrow M \longrightarrow R,$$

showing how a stakeholder concern is transformed into a structured test specified by Experimental Frame (E); how models (Ms) are selected and interpreted under that test, and how results (R) are made traceable back to the original question (Q). The example is intentionally minimal, but it captures the essential ideas that scale to more complex systems, including hybrid, multiformalism, and mission-level models. The formal DEVS definitions of the frames and models appear in Appendices A and B, while the main text focuses on the conceptual and methodological insights they support.

Formal Definitions of the Basic Relations

Having introduced the conceptual structure of EFSUT and motivated the need for a principled method to identify the minimal model capable of supporting a given Experimental Frame, we now turn to the formal definitions that ground these ideas. The response-time example provides an ideal entry point: its frames and models are simple enough to present in full DEVS form yet rich enough to illustrate the derivability, morphism, and applicability relations that drive the methodology. By specifying each Experimental Frame as a coupled DEVS model and each System Under Test as an atomic DEVS model, we make explicit the observational commitments, behavioral assumptions, and structural correspondences that the EFSUT workflow relies upon.

5. Relations Organizing Frames and Models in the EFSUT Methodology

The EFSUT methodology extends the DEVS formalism by introducing three relations—*derivability*, *applicability*, and *morphism*—that organize the space of Experimental Frames (EFs) and models (SUTs). These relations provide the structural foundation for transforming a stakeholder question into a well-defined test and for interpreting model behavior within that test. This section formalizes each relation and illustrates its role using the response-time example developed earlier (Figure 2).

The derivability, applicability, and morphism relations formalized here build on prior work on compatibility, abstraction, and partial-model organization [4,5,7,11]. In the following, we review these definitions at a higher level of analysis focusing on their application to the EFSUT methodology. Precise definitions are given in [6], and as stated in the section on future work, research is needed to further operationalize these concepts to support more automated generation from user question-based input.

An *Experimental Frame E* is a DEVS-coupled model specifying the admissible input segments, observable outputs, termination conditions, and interpretation rules under which a System Under Test (SUT) is evaluated.

A *model M* is a DEVS atomic system $M = \langle X, S, Y, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ representing the behavior of the SUT independently of any particular EF.

The three relations are summarized in Table 1, which presents their domains, formal definitions, and roles in the development and testing process.

5.1. Derivability: Structuring the Space of Tests

Definition 1 (Derivability)**.** *Let E and E′ be Experimental Frames. We say that E is* derivable *from E′ and write $E \leq E'$ if E can be obtained from E′ by a finite composition of the following:*

1. *Selection of compare variables;*
2. *Restriction of admissible input/output behaviors;*
3. *Coarsening of observable quantities.*

Theorem 1 (Frame Partial Order)**.** *The relation $\leq$ is a partial order on the set of Experimental Frames.*

In the response-time example, derivability yields the following hierarchy:

$$E_{\text{RT_passfail}} \leq E_{\text{RT}} \leq E_{\text{fullIO}}.$$

This ordering reflects the refinement of the original question into increasingly specific tests. The most inclusive frame E_{fullIO} records all I/O behavior without constraints. The frame E_{RT} restricts the test to a single input X, a single output Y, and a bounded observation window $T_{\max}$. Finally, $E_{\text{RT_passfail}}$ coarsens the observable by discarding the numeric response time. Using Hasse diagrams [12], we represent such partial orders using their *cover* relations and we interpret such representations as the transitive reduction of the EF derivability graph [13]. The resulting structures capture the minimal observational requirements needed to answer a given question.

5.2. Applicability: Determining Which Models Can Be Tested

Definition 2 (Applicability)**.** *An Experimental Frame E is* applicable *to a model M if the EF's admissible input segments, output map, and validity conditions can be meaningfully interpreted on M. This requires compatible input/output ports and interpretable EF semantics. For the converse relation, we say that a model accommodates a frame if, and only if, the frame is applicable to the model.*

In the response-time example, all three models—the deterministic delay model M_1, the stochastic delay model M_2, and the unbounded delay model M_3—share the same input/output ports as all three frames. Thus,

$$E_{\text{fullIO}}, E_{\text{RT}}, E_{\text{RT_passfail}} \text{ are applicable to } M_1, M_2, M_3.$$

Note that applicability establishes whether a model can answer questions that motivated an EF. But models are not constrained to the answers they give. For example, under the bounded-time frames:

- M_1 outputs pass or fail depending on whether $d \leq T_{\max}$;
- M_2 outputs pass or fail probabilistically;
- M_3 never produces output.

5.3. Morphism: Relating Models Across Abstraction Levels

Definition 3 (Model Morphism)**.** *Let M and M' be DEVS models. A* morphism $M \to M'$ *is a structure-preserving mapping that ensures the behavior of M' simulates or approximates the behavior of M at a specified level of abstraction.*

In the response-time example,

$$M_1 \to M_2 \quad (\sigma = 0, \mu = d),$$

meaning that the deterministic delay model is an exact special case of the stochastic delay model. Conversely,

$$M_2 \to M_1,$$

when the stochastic delay is approximated by a representative deterministic value (e.g., mean or worst-case delay). No useful morphisms involve M_3, whose behavior (never responding) is qualitatively different.

5.4. The EFSUT Chain

The three relations coordinate the entire development and testing workflow:

$$Q \longrightarrow E \longrightarrow M \longrightarrow R.$$

Each arrow is justified by the relations defined above. This chain ensures that simulation results are traceable to the original question, grounded in a well-defined EF, and interpreted in the context of the model's structural relationships. This is the core value of EFSUT: it makes the entire modeling and testing process explicit, organized, and defensible.

5.5. Identifying Models That Accommodate an Experimental Frame

In this section, we formulate a concept that enhances the EFSUT conciseness of presentation and ability to identify minimality of models vis a vis the questions they are intended to address. The key insight is that applicability propagates downward through the frame partial order. Formally, if a frame is derivable from a (more demanding) frame, and the latter is applicable to a model M, then the first frame is also applicable to the model. This inferred applicability assertion allows us to draw only the minimal applicability edges in a graph representation of the relations [12]. The resulting structure, shown in Table 2, captures the minimal observational requirements needed to answer a given question [13]. Further, by augmenting the representation with proper morphic-image arrows—reversing non-isomorphic morphisms—we obtain a compact representation that enables the modeler to determine the minimal model capable of answering the questions posed in any given frame. The resulting diagram supports a simple and transparent reasoning pattern: trace upward through derivability to find a more detailed frame with an explicit applicability edge; move across to the corresponding model; and trace downward through proper morphic-image arrows to identify the minimal applicable model. The procedure is implemented employing well-known graph algorithms [12,13].

Table 2. Minimal EFSUT relations and their formal basis.

Basic Relation	Justification (Grounded in Appendix C Definitions)
E_RT_passfail → E_RT	Basic derivability cover: E_RT_passfail is obtained from E_RT by coarsening the observable (replacing rt(t) with pass/fail)
E_RT → E_fullIO	Basic derivability cover. E_RT restricts the admissible behaviors of E_fullIO by emitting X at time 0 and imposing a bounded waiting window for Y. This is a refinement via restriction
E_fullIO → M1	Minimal applicability. E_fullIO is the most demanding frame, so its applicability is not inferred. Ports match and EF semantics (logging the input and output segments) are interpretable on M1.
E_fullIO → M2	Applicability of E_RT and E_RT_passfail is inferred, so E_fullIO → M2 is the only minimal edge.
E_fullIO → M3	Applicability does not require the model to satisfy the question—only that EF semantics are interpretable.
M1 → M2	Primitive morphism.M1 is an exact special case of M2 when the stochastic delay collapses to a point mass.
M2 → M1	Primitive morphism. Mapping a stochastic delay to a representative deterministic delay (mean, mode, worst-case) is an approximate morphism. No intermediate model exists, so it is Hasse-minimal.

6. Cross-Domain EFSUT Analysis: Questions and Experimental Frame Partial Order

6.1. Motivating Questions and Cross-Domain Framing

In the response-time example, we showed how EFSUT begins with a *question*—such as Q_{RT}—and generates a hierarchy of Experimental Frames (EFs) that differ only in the granularity of the observable variables they retain. We now apply the same methodology to two external DEVS-based systems: (i) the naval susceptibility DEVS architecture of [14] and (ii) the DSDEVS swarm system of [15]. Each system is driven by a domain-specific question that induces its own EF family, and both are unified by a cross-domain question that motivates an EF expressing combined Measures of Effectiveness (MOEs).

6.1.1. Naval Susceptibility Question

The naval paper [14] explicitly aims to evaluate susceptibility as a dynamic, stochastic, engagement-level process. (Here, susceptibility is understood generally as the probability that a ship will be hit by a weapon system given that a threat engagement has occurred.) This motivates the following question:

> $Q_{\mathbf{NAV_ETS}}$**:** *How does a naval surface ship's susceptibility evolve over time in a realistic, multi-threat engagement where detection, allocation, guidance, countermeasures, and environmental factors interact stochastically?*

The authors [14] describe their framework as "a simulation-based methodology for evaluating the susceptibility of naval surface ships in realistic engagement environments" and formalize susceptibility as "a dynamic probability redistribution process linking detection, selection, and allocation stages." This question induces the naval EF chain from full I/O logging to chain-level probability tracking to terminal susceptibility indicators.

6.1.2. Swarm DSDEVS Question

The swarm paper [15] emphasizes dynamic structure, adaptive behavior, and coverage as the primary performance measure. This motivates the following question:

> $Q_{\mathbf{SWARM_COV}}$**:** *How does a UAV swarm maintain or recover mission-level coverage under dynamic structural changes (add/remove UAVs), interference, damage, and environment-driven adaptation?*

The authors [15] state that "the UAV swarm model is a dynamic structural change model" and that the system must "continuously optimize and adjust its own behavior and structure." This question induces the swarm EF chain from full DSDEVS event logging to structural-change tracking to coverage/overlap MOEs.

6.1.3. Cross-Domain Question

Both domains share a common structure: DEVS/DSDEVS models producing time-resolved, event-driven stochastic behaviors whose performance is summarized through MOEs. This motivates a unifying question:

> $Q_{\mathbf{COMB_MOE}}$**:** *Across heterogeneous DEVS-based systems, how do system-level Measures of Effectiveness (MOEs) evolve over time under stochastic events, adaptive behaviors, and scenario-driven uncertainty?*

This question induces a combined MOE-level EF that serves as a meet between the naval and swarm full I/O frames.

6.2. Hasse-Reduced EF Partial Order with Paper-Grounded Justifications

The three questions above induce a set of Experimental Frames whose derivability relations form a combined EF partial order (Figure 3). Each Hasse-reduced arrow in the figure is listed with a justification tied explicitly to descriptions from the corresponding paper.

- $E_{\mathbf{NAV_chain}} \rightarrow E_{\mathbf{NAV_fullIO}}$
  Justification: ref. [14] describe the naval DEVS architecture as exposing full engagement-level event traces: "The overall system is structured as a multi-layered architecture composed of four major components: Adapter, Simulation, Modeler, and Data." The chain-level EF retains only the probabilistic redistribution variables (e.g., P_D, P_{HK}, P_{SK}), which is consistent with their statement that the framework "formalize[s] susceptibility as a dynamic probability redistribution process." Thus, $E_{\mathrm{NAV_chain}}$ is a coarsening of $E_{\mathrm{NAV_fullIO}}$.
- $E_{\mathbf{NAV_ETS}} \rightarrow E_{\mathbf{NAV_chain}}$
  Justification: ETS/STS are terminal indicators aggregated from the chain variables. The authors [14] emphasize that their methodology "presents a simulation-based method-

ology for evaluating the susceptibility of naval surface ships" and that susceptibility is expressed through the redistribution chain. Thus, $E_{\text{NAV_ETS}}$ is a further coarsening of $E_{\text{NAV_chain}}$.

- $E_{\textbf{SWARM_dynstruct}} \rightarrow E_{\textbf{SWARM_fullIO}}$
  Justification: ref. [15] state, "In summary, we use DSDEVS to model and simulate unmanned swarm systems," and further, "The UAV swarm model is a dynamic structural change model. The dynamic structure controller model adjusts the structure of the swarm model during the simulation operation." The dynamic-structure EF retains only structural changes, making it derivable from the full I/O EF.
- $E_{\textbf{SWARM_cov}} \rightarrow E_{\textbf{SWARM_dynstruct}}$
  Justification: Coverage and overlap are the primary MOEs reported in the experiment: "At simulation time 50. . . coverage rate of 91.58%. . . overlap rate 53.75%." Later: "At simulation time 70. . . overlap rate 20.37%. . . coverage rate 90.98%." These MOEs are aggregated from the structural events in $E_{\text{SWARM_dynstruct}}$, making the arrow Hasse-minimal.

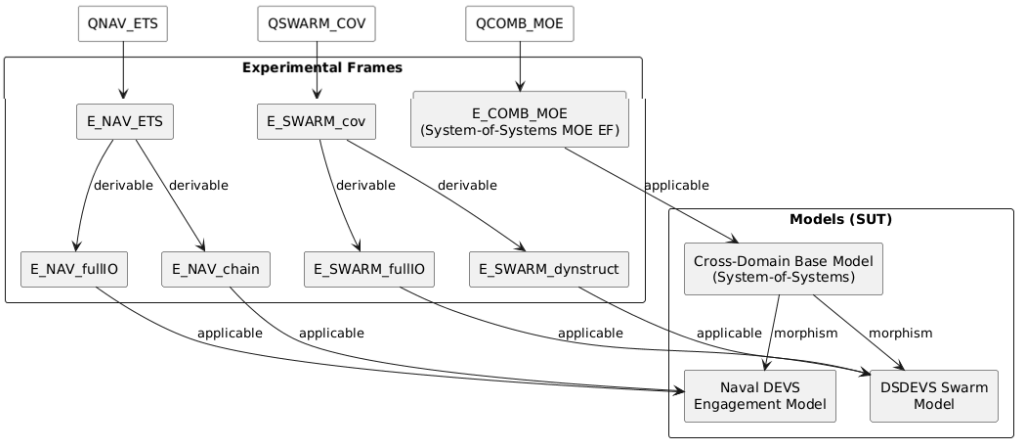


Figure 3. Combined EFSUT partial order for naval cross-domain synthesis that includes $E_{\text{COMB_MOE}}$.

6.3. Introducing the Combined EF That Requires a System-of-Systems Model

We introduce a combined Experimental Frame $E_{\text{COMB_MOE}}$ that fundamentally changes the observational requirements of the EFSUT structure. Unlike the domain-specific frames derived from $Q_{\text{NAV_ETS}}$ and $Q_{\text{SWARM_COV}}$, the combined question $Q_{\text{COMB_MOE}}$ asks for *system-of-systems* (SOS) Measures of Effectiveness (MOEs) that depend jointly on observables from both the naval engagement model and the swarm DSDEVS model.

These MOEs include, for example, the following:

- Joint survivability–coverage tradeoffs;
- Coordinated naval–swarm mission success probabilities;
- Cross-domain resource-allocation efficiency; and
- Susceptibility $\times$ reconnaissance persistence metrics.

None of these quantities can be computed from either model in isolation. The naval DEVS model provides detailed detection, allocation, guidance, and countermeasure events, but it has no representation of swarm-level coverage or dynamic structure. Conversely, the swarm DSDEVS model provides coverage, overlap, interference, and structural adaptation events, but it has no representation of naval susceptibility or kill-chain dynamics.

Thus, the combined EF $E_{\text{COMB_MOE}}$ requires a *joint* SUT that integrates the event streams of both domains. We introduce a notional *base model* representing such a system-of-systems integration layer. This model is the only SUT capable of satisfying the observational requirements of $E_{\text{COMB_MOE}}$.

To reflect this, $E_{\text{COMB_MOE}}$ applies exclusively to the base model and not to either domain-specific model. Furthermore, morphism arrows from the base model to the naval and swarm models indicate that the base model structurally contains and refines both domain models, enabling the combined EF to interpret cross-domain MOEs while preserving the semantics of each subsystem.

The resulting partial order of Figure 3 contains two domain-specific EF chains and a cross-domain MOE EF that unifies them. This structure supports a coherent, EFSUT-consistent synthesis of heterogeneous DEVS-based systems under a shared methodological framework.

6.4. Cross-Domain Applicability of Experimental Frames

Table 3 summarizes the applicability of each Experimental Frame (EF) to the two heterogeneous DEVS-based systems: (i) the naval susceptibility DEVS engagement model and (ii) the DSDEVS swarm model.

Table 3. Cross-domain applicability of Experimental Frames to the naval, swarm, and SoS base models.

Experimental Frame	Naval DEVS	Swarm DSDEVS	SoS Base Model	Justification (Paper-Grounded)
$E_{\text{NAV_fullIO}}$	Applicable	Not applicable	Not applicable	Naval model exposes full engagement-level events: "The overall system is structured as a multi-layered architecture…" Swarm and SoS models do not expose these naval-specific ports.
$E_{\text{NAV_chain}}$	Applicable	Not applicable	Not applicable	Chain variables (P_D, P_{HK}, P_{SK}) arise only in the naval kill-chain redistribution process.
$E_{\text{NAV_ETS}}$	Applicable	Not applicable	Not applicable	ETS/STS are naval-specific terminal susceptibility indicators.
$E_{\text{SWARM_fullIO}}$	Not applicable	Applicable	Not applicable	Swarm model exposes DSDEVS component-level events and dynamic structure: "The UAV swarm model is a dynamic structural change model…"
$E_{\text{SWARM_dynstruct}}$	Not applicable	Applicable	Not applicable	Structural-change events (add/remove UAV, controller decisions) exist only in the swarm DSDEVS model.
$E_{\text{SWARM_cov}}$	Not applicable	Applicable	Not applicable	Coverage/overlap MOEs are defined only for the swarm domain: "coverage rate of 91.58%… overlap rate 53.75%."
$E_{\text{COMB_MOE}}$	Not applicable	Not applicable	Applicable	Combined EF requires joint SoS-level observables (e.g., susceptibility × coverage tradeoffs) that cannot be computed from either model alone. Only the base model integrates both naval and swarm event streams and supports cross-domain MOEs.

Applicability follows the standard EFSUT criterion: an EF is applicable to a model if the model's observable ports and event semantics are sufficient to interpret the EF's required observables.

7. Cross-Domain Base Model Construction and Verification

In this section, we demonstrate how the earlier synthesis of Experimental Frames pays off in practice. We will show how a derived EF partial order directly drives the construction of a coherent model family that can accommodate the combined observational commitments. It is outside the scope of this paper to detail the construction process, so we point the reader to some references that discuss this process in detail [16–20].

7.1. Constructing the Cross-Domain Base Model Using DEVS Hierarchical Modular Composition

The cross-domain base model required by the combined Experimental Frame is constructed using standard DEVS hierarchical modular composition illustrated in Figure 4. Each domain model—the naval susceptibility DEVS engagement model and the dynamic-structure UAV swarm DSDEVS model—already exists as a well-formed coupled model with its own atomic components, ports, and event semantics. The base model is introduced as a higher-level DEVS coupled model whose purpose is to integrate these two heteroge-

neous subsystems and expose the joint observables required by the system-of-systems (SoS) combined EF.

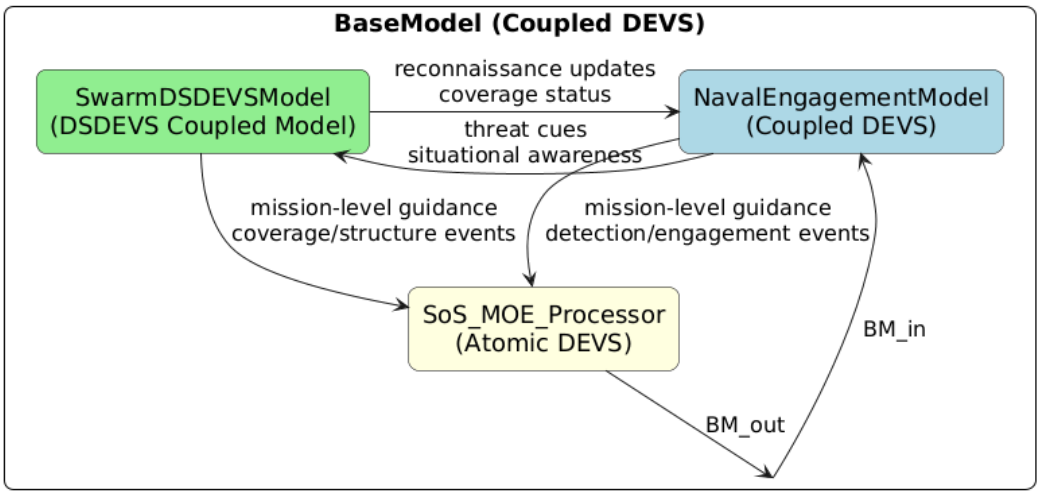


Figure 4. Cross-domain base coupled model.

This construction leverages the foundational DEVS properties of universality, uniqueness, and closure under coupling, which guarantee that heterogeneous DEVS-like systems can be composed into a single coherent model without violating behavioral semantics [21]. Closure under coupling, in particular, ensures that the naval and swarm models can be embedded as submodels within a larger coupled model while preserving their internal dynamics and enabling new cross-domain interactions.

At the structural level, the base model contains the naval and swarm models as subcomponents, which are connected through explicit DEVS couplings that represent cross-domain information flows. These couplings may include time-stamped messages representing shared situational awareness, reconnaissance updates, threat cues, or mission-level coordination signals. Because DEVS supports hierarchical modular construction, the base model encapsulates both domain models without altering their internal behavior while enabling new cross-domain interactions through external input and output ports.

The base model also introduces SoS-level atomic components responsible for computing joint Measures of Effectiveness (MOEs). These components subscribe to event streams from both submodels and maintain derived state variables such as combined susceptibility–coverage metrics, coordinated mission-success indicators, and cross-domain resource-allocation efficiency. In DEVS terms, these are standard atomic models whose input ports receive fused naval and swarm events and whose output ports expose the SoS-level observables required by the combined EF.

Finally, we note that it is not necessary that the base model include a synchronization semantics so that naval and swarm events can be interpreted within a common temporal frame. The DEVS coordinator assigned to the coupled model intrinsically manages the event occurrences of its component simulators. This ensures that the combined EF can observe cross-domain MOEs that depend on the relative timing of detection, engagement, coverage, and structural-change events. Because DEVS supports both modularity and compositional time advance, the base model integrates heterogeneous event streams without violating the semantics of either subsystem.

7.2. Verification of the Base Model for Combined-EF Applicability

To ensure that the base model accommodates the combined Experimental Frame, we apply the standard EFSUT applicability criterion: an EF is applicable to a model only if the

model exposes all observables required to interpret the EF's assertions. For the combined EF, these observables include joint susceptibility—coverage metrics, coordinated mission-success indicators, and other SoS-level MOEs that depend on fused naval and swarm event streams. Verification therefore begins by confirming that the base model's DEVS interface includes the necessary input and output ports, that its internal couplings correctly route naval and swarm events to the SoS-level MOE components, and that these components produce the derived cross-domain observables specified by the EF.

This verification process aligns with established DEVS verification practices. As summarized by Olamide et al. [22], DEVS model verification emphasizes interface conformance, hierarchical consistency, and compositional correctness. Interface conformance ensures that each coupled model exposes ports and message types consistent with its intended role. Hierarchical consistency ensures that the base model preserves the behavioral semantics of its constituent naval and swarm models when embedded as subcomponents. Compositional correctness ensures that the addition of SoS-level MOE components and cross-domain couplings does not violate DEVS closure under coupling or introduce unintended emergent behavior.

Finally, verification checks that the morphism mappings from the base model to the naval and swarm models preserve subsystem semantics while enabling the fused observables required by the combined EF. Once these structural and behavioral conditions are satisfied, the combined EF is formally applicable to the base model.

In short, the combined Experimental Frame ($E_{\text{COMB_MOE}}$) in Table 3 applies exclusively to the system-of-systems (SoS) base model, rather than to the individual Naval DEVS or Swarm DSDEVS models. While this exclusivity might seem to make the methodology dependent on a specific combined model, the reverse is true. In fact, $E_{\text{COMB_MOE}}$ is derived from and constrained by the component EFs, which dictate model capabilities. Because multiple model architectures—from minimal realizations to performance-optimized variations—can satisfy a single EF, the core principle remains: the Experimental Frame must precede and constrain the models, not vice versa.

8. Discussion: Positioning EFSUT Among Verification Frameworks

To clarify the contribution of this paper, the main contributions are summarized in Table 4.

Table 4. Summary of main contributions.

Contribution	What It Adds	Role Played in EFSUT Methodology
1. Unified methodological pipeline	Establishes the end to end chain from questions → frames → models → mission threads.	Provides the structural backbone that links stakeholder intent to executable simulation artifacts and mission thread reasoning.
2. Principled organization of frames and models	Defines how derivability, applicability, and morphism relations jointly structure the test/model space.	Ensures traceability, justified abstraction, and minimal model identification across heterogeneous modeling contexts.
3. Compact partial order Hasse-transitivity representation	Introduces compact subrelations capturing frame cover relations, applicability edges, and model morphisms.	Makes the relational structure interpretable, enabling transparent reasoning about model choice, abstraction, and adequacy.
4. Minimal model identification procedure	Provides a systematic method for selecting the simplest model capable of supporting a given frame.	Reduces modeling effort, prevents unnecessary complexity, and enforces the "minimal sufficient model" principle.
5. Demonstration across simple and complex contexts	Shows EFSUT applied to both a simple timing example and a multilevel, multiformalism ecosystem.	Demonstrates scalability from basic DEVS reasoning to realistic system-of-systems analysis.

Verification and validation frameworks across systems engineering, software testing, and model-based design share a common goal: to ensure that models and implementations behave as intended. However, they differ significantly in how they define correctness, how they structure test specifications, and how they relate stakeholder intent to model behavior. The EFSUT methodology occupies a distinctive position within this landscape by explicitly organizing the relationships among questions, Experimental Frames, and models through the three relations summarized in Table 1. This section discusses how EFSUT compares to and complements other verification approaches.

1. *EFSUT vs. Traditional DEVS-Based Verification* Classical DEVS verification [10,23] focuses on internal model correctness—closure under coupling, determinism, well-formed transition functions, and conformance to the DEVS abstract simulator. While these checks ensure that a model is structurally sound, they do not address whether the model is appropriate for a particular stakeholder concern. DEVS provides the machinery for simulation but not the methodology for determining what should be simulated or how results should be interpreted [24].

EFSUT extends DEVS by introducing the following:

- Questions as motivating the development of experimental frames, which in turn can support the development of accommodating models,
- Derivability to structure the refinement of questions into executable frames;
- Applicability to determine when a frame can be meaningfully applied to a model; and
- Morphism to relate models across abstraction levels.

In this way, EFSUT complements DEVS by providing a methodological layer that connects model correctness to test relevance.

2. *EFSUT vs. SES-Driven Model Families* The System Entity Structure provides a powerful mechanism for representing model families through decomposition, specialization, and configuration [25,26]. SES–DEVS integration enables systematic generation of model variants, but the SES does not prescribe how those variants should be evaluated or how stakeholder concerns should guide the selection of configurations.

EFSUT fills this methodological gap through the following:

- Using derivability to derive frames that correspond to specific stakeholder concerns;
- Using applicability to determine which SES-generated models are meaningful under those frames; and
- Using morphism to justify substituting simpler SES-generated models for more complex ones.

Thus, SES organizes the space of models, while EFSUT organizes the space of tests and the relationships between tests and models. Together, they form a coherent framework for model generation, selection, and evaluation.

3. *EFSUT vs. Model-Based Testing (MBT)* Model-based testing frameworks typically generate test cases from a behavioral model such as a state machine, automaton, or formal specification [27–32]. MBT excels at producing systematic test suites, but it generally assumes the following:

- A fixed test specification;
- A single model of reference; and
- A direct mapping from model behavior to test cases.

MBT does not provide the following:

- A formal mechanism for deriving test specifications from stakeholder concerns;
- A relational structure for determining when a test is meaningful for a given model; or
- A way to relate multiple models across abstraction levels.

EFSUT complements MBT by providing the following:

- Questions → the origin of test intent;
- Frames → the formal test specifications derived from questions;
- Applicability → the criterion for determining whether a test applies to a model;
- Morphism → the mechanism for relating multiple models used in MBT workflows.

In this sense, EFSUT can be viewed as a meta-framework that organizes and justifies the use of MBT within a broader systems-theory context.

4. *EFSUT vs. Formal Verification and Model Checking* Formal verification techniques such as model checking [33] provide exhaustive guarantees about system behavior with respect to temporal logic specifications [34]. These methods are powerful but require the following:

- A precise formal specification;
- A finite or finitely abstracted state space; and
- A model that is amenable to symbolic or explicit-state exploration.

EFSUT differs in several ways:

- It does not require a formal logic specification; instead, it derives testable frames from stakeholder questions.
- It supports simulation-based evaluation, which is applicable to systems with continuous dynamics, stochastic behavior, or unbounded state spaces.
- It provides a relational structure (derivability, applicability, morphism) for organizing tests and models even when formal verification is infeasible.

EFSUT therefore complements formal verification by providing a principled methodology for simulation-based reasoning in contexts where formal methods cannot be applied directly.

5. *EFSUT as a Bridge Between Modeling, Testing, and Decision Support* Across all these frameworks, EFSUT's distinctive contribution is its ability to achieve the following:

- Connect stakeholder intent to model evaluation;
- Structure the refinement of concerns into executable tests;
- Determine when tests are meaningful for particular models; and
- Relate models across abstraction levels in a principled way.

This relational structure makes EFSUT particularly well suited for the following:

- Multi-model workflows;
- Hierarchical model families;
- Simulation-based certification;
- Mission-level decision support; and

contexts where both qualitative and quantitative concerns must be integrated.

In these settings, EFSUT provides a xmethodological backbone that complements and enhances existing verification frameworks rather than replacing them.

9. Conclusions and Future Directions

This paper has defined the EFSUT methodology as a major synthesis of prior work on experimental frame design, relation to simulation models, multiformalism modeling, and model discovery [3,5–7,9,11]. In so doing, we have demonstrated that this methodology provides a principled, relational foundation for linking stakeholder questions to the models capable of answering them. By formalizing questions as Experimental Frames, organizing frames through derivability, structuring models through morphisms, and defining applicability as the interpretability of frame variables on model behavior, EFSUT establishes a transparent and auditable workflow from intent to simulation results. The response-time

example and the naval-swarm use case demonstrate that the same relational structure scales from minimal pedagogical settings to complex scientific modeling environments. Across these examples, the methodology consistently identifies minimal applicable models, prevents unnecessary model complexity, and ensures that simulation results remain traceable to the observational commitments that justify them.

That work demonstrates how the constructs developed here provide the methodological backbone for lifecycle-consistent digital-twin practice, including model adequacy management, drift detection, recalibration workflows, and continuous assurance. In that context, EFSUT becomes not only a framework for principled model selection but also a unifying scaffold for integrating sensing, prediction, validation, and decision support across the digital-twin lifecycle.

Continued research is needed to build on this foundation. A companion article [35] extends these ideas into the domain of digital engineering and digital twins. Other promising directions include the automated derivation of Experimental Frames from requirements artifacts, integration with MBSE tool-chains, formal verification of frame–model applicability, and scalable simulation-based decision pipelines [31,32,36,37]. As systems grow in complexity and digital engineering becomes increasingly central to lifecycle management [38–43], the need for principled, traceable, and minimal modeling methodologies will only intensify. EFSUT provides a rigorous foundation for meeting that need. As an example, work is in progress to extend the X language with first-class constructs for observation commitments, experimental frames, and applicability relations, embedding them directly into the modeling syntax [44]. These extensions should be aimed at unifying requirements, architecture, and DEVS-based simulation semantics, enabling automated traceability from stakeholder intent to executable models and ensuring minimal sufficient model selection during verification.

Future research should more explicitly articulate the non-technical gap that EFSUT fills beyond its role in simulation execution—namely, its ability to unify intent, observability, and traceability across the entire system lifecycle. Traditional requirements engineering relies on static cross-reference matrices to map verification activities to predefined requirements, but these artifacts rarely evolve once a system enters operation, sustainment, or redesign phases. In contrast, EFSUT treats experimental frames as dynamic, question-driven constructs that can be re-instantiated to support adaptive evaluation even when formal requirements are incomplete, outdated, or absent. This opens several promising research directions: applying EFSUT to operational analytics for in-service systems, using EFSUT to guide system optimization and redesign under evolving mission conditions, and extending EFSUT to non-directed or emergent system-of-systems contexts where development is not driven by detailed specifications. Investigating these directions would demonstrate EFSUT's broader value as a lifecycle-spanning framework that bridges the gap between requirement-driven verification and question-driven experimentation. Furthermore, by treating questions as intentional precursors of executable experimental frames, this approach opens a broader research path toward formalizing questions as first-class lifecycle artifacts that support adaptive evaluation, redesign, and system-of-systems reasoning.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/systems14060691/s1. Because of its extensive elucidation of experimental frame concepts and in-depth application, the book chapter in Reference [6] has been made available for open access with the publisher's permission at DOI: 10.1016/B978-0-12-318750-5.50007-6.

Funding: This research received no external funding

Data Availability Statement: The original contributions presented in this study are included in the article/Supplementary Materials. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The author is chief scientist with a company that commercializes software related to the content of this article.

Appendix A. Detailed DEVS Definitions for the Response-Time EFSUT Example

This appendix provides expanded explanations of the DEVS models used in the response-time example. The goal is to show not only the formal definitions but also how each component supports the EFSUT methodology.

Appendix A.1. Atomic DEVS Models (Systems Under Test)

In EFSUT, the System Under Test (SUT) is always represented as a DEVS atomic model. The EF constrains how the SUT is stimulated and observed, but the SUT itself is defined independently of any particular test. This separation is essential: it allows the same model to be evaluated under different questions and frames.

We consider three timing models, which are each representing a different behavioral assumption about how long the system takes to respond to an input.

Appendix A.1.1. Deterministic Delay Model M_1

The deterministic model assumes that once the system receives input X, it will always produce output Y after a fixed delay d. This is the simplest timing behavior and serves as a baseline for comparison.

$$M_1 = \langle X, S, Y, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$$

Ports

$$X = \{(inX, v)\}, \qquad Y = \{(outY, v)\}.$$

The model accepts a single input event and produces a single output event.

State Set

$$S = \{idle\} \cup \{processing(d)\}.$$

The model is either waiting for input or processing it.

Time Advance

$$ta(idle) = \infty, \qquad ta(processing(d)) = d.$$

The system waits indefinitely for input, but once processing begins, the internal event is scheduled exactly d time units later.

Transitions and Output

$$\delta_{ext}(idle, e, inX) = processing(d),$$

$$\delta_{int}(processing(d)) = idle,$$

$$\lambda(processing(d)) = (outY, v).$$

Thus, the model always produces Y exactly d time units after X.

Appendix A.1.2. Stochastic Delay Model M_2

The stochastic model generalizes M_1 by allowing the delay to vary randomly. This captures systems whose timing behavior is influenced by noise, load, or other uncertainties.

The structure is identical to M_1 except for the external transition:

$$\theta \sim D(\mu, \sigma), \qquad \delta_{ext}(idle, e, inX) = processing(\theta).$$

The time advance is then $ta(processing(\theta)) = \theta$.

This model can produce both "pass" and "fail" outcomes under a bounded response-time EF, depending on the sampled delay.

Appendix A.1.3. Unbounded Delay Model M_3

The unbounded model represents systems that may accept input but never produce output. This is important in EFSUT because applicability does not require that a model satisfy the question—only that the EF can be interpreted on the model.

$$S = \{idle, processing\}, \qquad ta(idle) = ta(processing) = \infty.$$

There are no internal transitions and no outputs:

$$\lambda(s) \text{ undefined for all } s.$$

Appendix B. DEVS Definitions of the Experimental Frames

This appendix provides DEVS specifications for the three Experimental Frames used in the response-time example. Each EF is expressed as a coupled model composed of a *Generator*, *Acceptor*, and *Transducer*, following the standard DEVS EF architecture. These definitions parallel the SUT models given earlier and complete the formal description of the EFSUT test chain.

Appendix B.1. Full I/O Frame E_{fullIO}

The full I/O frame observes all inputs and outputs of the SUT without imposing timing constraints or termination conditions.

Definition A1 (Full I/O Frame)**.** *The Experimental Frame*

$$E_{fullIO} = \langle X_E, Y_E, D, \{M_d\}, Select \rangle$$

is defined as a DEVS coupled model with the following:

- $X_E = \{ start \}$ *(external trigger to begin observation);*
- $Y_E = \{ log(x), log(y) \}$ *(logged observations);*
- $D = \{Gen, Acc, Trans\}$.

Each component is defined as follows:

Generator M_{Gen}

$$M_{Gen} = \langle X, S, Y, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$$

where it emits no inputs to the SUT (pure observer):

$$Y = \emptyset, \quad ta(s) = \infty.$$

Acceptor M_{Acc}

passively forwards all SUT outputs to the transducer

$$X = Y_{SUT}, \quad Y = \{log(y)\}.$$

Transducer M_{Trans}

records all events:

$$X = \{log(x), log(y)\}, \quad Y = \emptyset.$$

Couplings

$$SUT.out \rightarrow Acc.in, \qquad Acc.out \rightarrow Trans.in.$$

Appendix B.2. Response-Time Frame E_{RT}

This frame emits a single input X at time 0 and waits up to $T_{\max}$ for the corresponding output Y, measuring the elapsed time.

Definition A2 (Response-Time Frame)**.** *The Experimental Frame*

$$E_{RT} = \langle X_E, Y_E, D, \{M_d\}, Select \rangle$$

is defined with

$$X_E = \{start\}, \qquad Y_E = \{rt(t)\}.$$

Generator M_{Gen}

emits a single input X *at time* 0*:*

$$S = \{s_0, s_1\}, \quad ta(s_0) = 0, \quad ta(s_1) = \infty,$$

$$\lambda(s_0) = X.$$

Acceptor M_{Acc}

passively receives the SUT output Y*:*

$$X = \{Y\}, \quad Y = \{Y\}.$$

Transducer M_{Trans}

measures response time:

$$S = \{(t_{start}, t_{resp})\},$$

$$X = \{X, Y\}, \quad Y = \{rt(t)\}.$$

On receiving X *at time* 0*, it records* $t_{start} = 0$*. On receiving* Y *at time* t*, it outputs* $rt(t)$ *and transitions to a terminal state.*

Termination

The frame terminates at $\min(t_{resp}, T_{\max})$.

https://doi.org/10.3390/systems14060691

Appendix B.3. Pass/Fail Response-Time Frame $E_{RT\text{-}passfail}$

This frame coarsens the observation: instead of reporting the measured delay, it outputs a Boolean indicating whether the response occurred before $T_{\max}$.

Definition A3 (Pass/Fail Response-Time Frame)**.** *The Experimental Frame*

$$E_{RT\text{-}passfail} = \langle X_E, Y_E, D, \{M_d\}, Select \rangle$$

is defined with

$$X_E = \{start\}, \qquad Y_E = \{pass, fail\}.$$

Generator M_{Gen}

is identical to E_{RT}:

$$\lambda(s_0) = X, \qquad ta(s_0) = 0.$$

Acceptor M_{Acc}

identical to E_{RT}.

Transducer M_{Trans}

coarsens the following observation:

$$X = \{X, Y\}, \quad Y = \{pass, fail\}.$$

Under bounded

Appendix C. Mathematical Appendix: DEVS Definitions for EFSUT Example

Appendix C.1. Atomic DEVS Models

Appendix C.1.1. Deterministic Delay Model M_1

$$M_1 = \langle X, S, Y, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$$

$$X = \{(inX, v)\}, \quad Y = \{(outY, v)\},$$

$$S = \{idle\} \cup \{processing(d)\},$$

$$ta(idle) = \infty, \quad ta(processing(d)) = d,$$

$$\delta_{ext}(idle, e, inX) = processing(d),$$

$$\delta_{int}(processing(d)) = idle,$$

$$\lambda(processing(d)) = (outY, v).$$

Appendix C.1.2. Stochastic Delay Model M_2

This is the same as M_1 except for

$$\theta \sim D(\mu, \sigma), \qquad \delta_{ext}(idle, e, inX) = processing(\theta).$$

Appendix C.1.3. Unbounded Delay Model M_3

$$S = \{idle, processing\}, \quad ta(idle) = ta(processing) = \infty,$$

$$\delta_{ext}(idle, e, inX) = processing,$$

No internal transitions; no outputs.

Appendix C.2. Experimental Frames

Appendix C.2.1. Full I/O Frame E_{fullIO}

$$E_{\text{fullIO}} = \langle X_E = \emptyset,\ Y_E,\ D = \{GEN, TRN, SUT\},\ EIC, EOC, IC \rangle.$$

GEN emits any value in V_X; TRN accepts any value in V_Y.

Appendix C.2.2. Response-Time Frame E_{RT}

GEN emits X at time 0; TRN waits up to $T_{\max}$ and outputs pass/fail with Δt.

Appendix C.2.3. Pass/Fail Frame $E_{\text{RT_passfail}}$

Coarsened version of E_{RT}.

Appendix C.3. Relations

Appendix C.3.1. Derivability

$$E \leq E' \iff E \text{ is derivable from } E'.$$

$$E_{\text{RT_passfail}} \leq E_{\text{RT}} \leq E_{\text{fullIO}}.$$

Appendix C.3.2. Applicability

$$E \text{ applicable to } M \iff \text{ports match and EF semantics interpretable.}$$

Thus,

$$E_{\text{fullIO}}, E_{\text{RT}}, E_{\text{RT_passfail}} \text{ applicable to } M_1, M_2, M_3.$$

Appendix C.3.3. Morphism

$$M_1 \to M_2 \quad (\sigma = 0, \mu = d),$$

$$M_2 \dashrightarrow M_1.$$

Appendix C.4. EFSUT Chain

$$Q \longrightarrow E \longrightarrow M \longrightarrow R.$$

References

1. Zeigler, B.P.; Muzy, A.; Kofman, E. *Theory of Modeling and Simulation*, 3rd ed.; Academic Press: London, UK, 2018.
2. Ören, T.; Neto, V.V.G. Reliability and Quality Assurance of M&S. In *Body of Knowledge for Modeling and Simulation: A Handbook by the Society for Modeling and Simulation International*; Ören, T., Zeigler, B.P., Tolk, A., Eds.; Springer: Cham, Switzerland, 2023; pp. 167–204. [CrossRef]
3. Chreyh, R.; Wainer, G.A. CD++ Repository: An Internet-Based Searchable Database of DEVS Models and Their Experimental Frames. In *SpringSim 2009 (DEVS Symposium)*; Society for Computer Simulation International: San Diego, CA, USA, 2009.
4. Ponnusamy, S.S.; Albert, V.; Thebault, P. Modeling & Simulation Framework for the Inclusion of Simulation Objectives by Abstraction. In *SIMULTECH 2014*; Science and Technology Publications, Lda: Setúbal, Portugal, 2014; pp. 385–394.
5. Traoré, M.K.; Muzy, A. Capturing the dual relationship between simulation models and their context. *Simul. Model. Pract. Theory* **2006**, *14*, 126–142. [CrossRef]
6. Zeigler, B.P. Multilevel Multiformalism Modeling: An Ecosystem Example. In *Theoretical Systems Ecology*; Academic Press: Cambridge, MA, USA, 1979; pp. 17–54. [CrossRef]
7. Zeigler, B.P. Structuring the Organization of Partial Models. *Int. J. Gen. Syst.* **1978**, *4*, 81–88. [CrossRef]
8. Cao, B.; Huang, L.; Hu, J. Experimental Frame Design Using E-DEVSML for Software Quality Evaluation. In Proceedings of the SEKE 2015, Pittsburgh, PA, USA, 6–8 July 2015; pp. 310–313. Available online: https://www.researchgate.net/publication/300561326_Experimental_Frame_Design_Using_E-DEVSML_for_Software_Quality_Evaluation (accessed on 16 April 2026).
9. Zeigler, B.P. A methodology to characterize simulation models for discovery and composition: A system theory-based approach to model curation for integration and reuse. *Int. J. Simul. Process Model.* **2022**, *19*, 3–13.
10. Traoré, M.K.; Gorecki, S.; Ducq, Y. A Formal Framework for Digital Twin Modeling, Verification, and Validation. In *Digital Twins, Simulation, and the Metaverse*; Grieves, M., Hua, E.Y., Eds.; Simulation Foundations, Methods and Applications; Springer: Cham, Switzerland, 2024.
11. Foures, D.; Albert, V.; Nketsa, A. Simulation Validation Using the Compatibility Between Simulation Model and Experimental Frame. In Proceedings of the SummerSim 2013, Toronto, ON, Canada, 7–10 July 2013; p. 75. Available online: https://hal.science/hal-00868406 (accessed on 16 April 2026).
12. Davey, B.A.; Priestley, H.A. *Introduction to Partial Orders and Order*, 2nd ed.; Cambridge University Press: Cambridge, UK, 2002.
13. Aho, A.V.; Garey, M.R.; Ullman, J.D. The transitive reduction of a directed graph. *SIAM J. Comput.* **1972**, *1*, 131–137. [CrossRef]
14. Byeon, G.W.; Oh, S.H.; Kwak, M.J.; Yoon, J.Y.; Han, W.; Woo, J.H. Development of a methodology for evaluating the susceptibility of naval surface ships based on naval engagement simulation. *Appl. Ocean Res.* **2025**, *165*, 104839. [CrossRef]
15. Zhang, W.; Li, Q.; Xu, X.; Li, W. Modeling and Simulation of Unmanned Swarm System Based on Dynamic Structure DEVS. *J. Phys. Conf. Ser.* **2024**, *2755*, 012021. [CrossRef]
16. DEVS Network. DEVS Tools. Available online: https://devs-network.org/tools/ (accessed on 1 January 2016).
17. Belloli, L.; Vicino, D.; Ruiz-Martin, C.; Wainer, G. Building DEVS models with the cadmium tool. In *WSC '19: Proceedings of the Winter Simulation Conference*; IEEE: Piscataway, NJ, USA, 2020; pp. 45–59.
18. Lee, C.H.; Bae, J.W.; Paik, E. GO-DEVS: Storage and Retrieval System for DEVS Models Using Graph and Ontology Representation. *Sensors* **2021**, *21*, 6771. [CrossRef] [PubMed]
19. Capocchi, L.; Santucci, J.-F.; Poggi, B.; Nicolai, C. DEVSimPy: A Collaborative Python Software for Modeling and Simulation of DEVS Systems. In Proceedings of the 20th IEEE International Workshops on Enabling Technologies: Infrastructures for Collaborative Enterprises (WETICE 2011), Paris, France, 27–29 June 2011; pp. 1–6. [CrossRef]
20. Blas, M.J.; Gonnet, S. Metamodel-based formalization of DEVS atomic models. *Simulation* **2023**, *99*, 433–454. [CrossRef]
21. Zeigler, B.P.; Kewley, R.; Wainer, G. DEVS Closure Under Coupling, Universality, and Uniqueness: Enabling Simulation and Software Interoperability from a System-Theoretic Foundation. *Computers* **2025**, *14*, 514. [CrossRef]
22. Olamide, S.E.; Kaba, T.M. Formal Verification and Validation of DEVS Simulation Models. In Proceedings of the AFRICON 2013, Pointe aux Piments, Mauritius, 9–12 September 2013; pp. 1–6.
23. Samuel, K.G.; Bouare, N.D.M.; Maïga, O.; Traoré, M.K. A DEVS-based pivotal modeling formalism and its verification and validation framework. *Simulation* **2020**, *96*, 969–992. [CrossRef]
24. Zeigler, B.P.; Nutaro, J. Towards a Framework for Computational Representation of Intended Use. *J. Simul.* **2016**, *10*, 1–15.
25. Junglas, P.; Jammer, D.; Pawletta, T.; Pawletta, S. Using component-based discrete-event modeling with NSA-DEVS—An invitation. In *Proceedings of the ASIM 2024 Tagungsband Langbeiträge, 27th Symposium Simulationstechnik*; ARGESIM-Verlag: Vienna, Austria, 2024.
26. Folkerts, H. An Architecture for Model Behavior Generation for Multiple Simulators. Ph.D. Thesis, University of Applied Sciences Wismar, Wismar, Germany, 2024.
27. Fortino, G.; Savaglio, C.; Spezzano, G.; Zhou, M. Internet of Things as System of Systems: A Review of Methodologies, Frameworks, Platforms, and Tools. *IEEE Trans. Syst. Man Cybern. Syst.* **2021**, *51*, 223–236. [CrossRef]
28. Alavi Fazel, I.; Wainer, G. Discrete Event System Specification for IoT Applications. *Sensors* **2024**, *24*, 7784. [CrossRef] [PubMed]

29. Sehili, S.; Capocchi, L.; Santucci, J.F.; Lavirotte, S.; Tigli, J.Y. Discrete Event Modeling and Simulation for IoT Efficient Design Combining WComp and DEVSimPy Framework. In *Proceedings of the 5th International Conference on Simulation and Modeling Methodologies, Technologies and Applications (SIMULTECH 2015)*; Science and Technology Publications, Lda: Setúbal, Portugal, 2015; pp. 26–34.
30. Zhang, L.; Zhao, C. *Modeling and Simulation Based Systems Engineering*; World Scientific: Singapore, 2023.
31. Ansari, A.M.; Nazir, M.; Mustafa, K. Ontology-Based Classification and Detection of the Smart Home Automation Rules Conflicts. *IEEE Access* **2024**, *12*, 85072–85088. [CrossRef]
32. Capocchi, L.; Santucci, J.F.; Tigli, J.Y.; Gomnin, T.; Lavirotte, S.; Rocher, G. Actuation Conflict Management in Internet of Things Systems DevOps: A Discrete Event Modeling and Simulation Approach. In *Proceedings of the Internet of Things (IFIP AICT 728)*; Springer: Cham, Switzerland, 2025; pp. 189–206.
33. Clarke, E.M., Jr.; Grumberg, O.; Kroening, D.; Peled, D.; Veith, H. *Model Checking*; MIT Press: Cambridge, MA, USA, 2018.
34. Fazel, I.A.; Wainer, G. A DEVS-Based Methodology for Simulation and Model-Driven Development of IoT. In *Simulation Tools and Techniques*; Springer Nature Switzerland AG: Cham, Switzerland, 2024; pp. 3–17. Available online: https://www.semanticscholar.org/paper/A-DEVS-Based-Methodology-for-Simulation-and-of-IoT-Fazel-Wainer/1468bfa2fd985093084d022933cd0755d488dbb9 (accessed on 16 April 2026).
35. Zeigler, B.P. Experimental Frame–System Under Test (EFSUT): A Principled Foundation for Model Choice and Lifecycle Management in Digital Twins. *Digit. Twins Digit. Eng. J.* **2026**, *4*, 1–26. [CrossRef]
36. Tendeloo, Y.V.; Vangheluwe, H. An evaluation of DEVS simulation tools. *Simulation* **2017**, *93*, 103–121.
37. Harr, N.; Fleurey, F.; Morin, B.; Husa, K.E. ThingML: A Language and Code Generation Framework for Heterogeneous Targets. In *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems (MODELS '16)*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 125–135. Available online: http://www.cs.toronto.edu/~chechik/courses18/csc2125/paper79.pdf (accessed on 16 April 2026).
38. Risco-Martín, J.L.; Mittal, S.; Fabero, J.C.; Malagón, P.; Ayala, J.L. Real-Time Hardware/Software Co-Design Using DEVS-Based Transparent M&S Framework. In Proceedings of the Summer Computer Simulation Conference (SCSC '16), Montreal, QC, Canada, 24–27 July 2016. Available online: https://www.sce.carleton.ca/faculty/wainer/papers/SCSC53.pdf (accessed on 16 April 2026).
39. Wymore, A.W. *A Mathematical Theory of Systems Engineering: The Elements*; Krieger: Huntington, NY, USA, 1967.
40. Reggio, G. A UML-based proposal for IoT system requirements specification. In *Proceedings of the 10th International Workshop on Modelling in Software Engineering (MiSE '18)*; ACM: New York, NY, USA, 2018; pp. 9–16.
41. Xiao, R.; Wu, Z.; Wang, D. A Finite-State-Machine model driven service composition architecture for internet of things rapid prototyping. *Future Gener. Comput. Syst.* **2019**, *99*, 473–488.
42. da Silva Fonseca, J.P.; de Sousa, A.R.; de Souza Tavares, J.J.-P.Z. Modeling and controlling IoT-based devices' behavior with high-level Petri nets. *Procedia Comput. Sci.* **2023**, *217*, 1462–1469. [CrossRef]
43. Escamilla-Ambrosio, P.J.; Robles-Ramírez, D.A.; Tryfonas, T.; Rodríguez-Mota, A.; Gallegos-García, G.; Salinas-Rosales, M. IoTsecM: A UML/SysML Extension for Internet of Things Security Modeling. *IEEE Access* **2021**, *9*, 154112–154135.
44. Xie, K.; Zhang, L.; Laili, Y.; Wang, X. XDEVS: A hybrid system modeling framework. *Int. J. Model. Simul. Sci. Comput.* **2022**, *13*, 2243001. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.